



Department of Computer Engineering
CENG350 Software Engineering
Software Architecture Description (SAD)
for KSO

Group 5

By

Alper Mehmet Çelebi, 2580405

Deniz Karakoyun, 2580678

Wednesday 14th May, 2025

Contents

List of Figures	iii
List of Tables	v
1 Introduction	vii
1.1 Purpose and objectives of KSO	vii
1.2 Scope	viii
1.3 Stakeholders and their concerns	ix
2 Glossary	xi
3 Architectural Views	xiv
3.1 Functional View	xiv
3.1.1 Stakeholders' Uses of This View	xiv
3.1.2 Component Diagram	xv
3.1.3 Internal Interfaces	xvii
3.1.4 Interaction Patterns	xx
3.1.5 External Interfaces	xxv
3.1.6 Interaction Scenarios	xxvii
3.2 Information View	xxix
3.2.1 Stakeholders' Uses of This View	xxix
3.2.2 Database Class Diagram	xxx
3.2.3 Operations on Data	xxx
3.3 Development View	xxxiii

3.3.1	Stakeholders' Uses of This View	xxxiii
3.3.2	Package Diagram	xxxiv
3.4	Deployment View	xxxviii
3.4.1	Stakeholders' Uses of This View	xxxviii
3.4.2	Deployment Diagram	xxxix
3.5	Design Rationale	xliv
3.5.1	Functional View Rationale	xliv
3.5.2	Information View Rationale	xliv
3.5.3	Development View Rationale	xliv
3.5.4	Deployment View Rationale	xlvi
4	Architectural Views for Your Suggestions to Improve the Existing System	xlvi
4.1	Functional View	xlvi
4.1.1	Stakeholders' Uses of This View	xlvi
4.1.2	Component Diagram	xlvi
4.1.3	Internal Interfaces	1
4.1.4	Interaction Patterns	1
4.1.5	External Interfaces	li
4.1.6	Interaction Scenarios	liii
4.1.7	Database Class Diagram	lv
4.1.8	Operations on Data	lvi
4.2	Development View	lvii
4.2.1	Stakeholders' Uses of This View	lvii
4.2.2	Package Diagram	lvii
4.3	Deployment View	lx
4.3.1	Stakeholders' Uses of This View	lx
4.3.2	Deployment Diagram	lx
4.4	Design Rationale	lxiv

List of Figures

3.1	Component diagram designed for KSO system.	xvi
3.2	Internal Interfaces Class Diagram of the KSO System	xxviii
3.3	Sequence diagram showing the annotation validation process between citizen scientists, the dashboard, and backend services.	xxi
3.4	Sequence diagram of the automated model retraining workflow.	xxiii
3.5	Sequence diagram of the distributed backup process following the 3-2-1 rule.	xxiv
3.6	External interfaces diagram of KSO system.	xxvi
3.7	Activity Diagram of Researcher-Guided Automated Annotation Workflow.	xxviii
3.8	Activity Diagram of Crowdsourced Annotation Validation Scenario. . .	xxix
3.9	Database class diagram of KSO system.	xxxi
3.10	Package diagram of the KSO System.	xxxv
3.11	Deployment diagram of the KSO System.	xxxix
4.1	Revised component diagram of the KSO system, extended with the <i>Active-Learning & Annotation Orchestrator (ALAO)</i>	xlix
4.2	Updated External Interfaces class diagram including the two new APIs (highlighted).	li
4.3	Activity diagram of the Active Learning feedback loop spanning ML Model Interface, ALAO, Zooniverse, and Data Management components.	liii
4.4	Updated database class diagram with new suggested entities highlighted in green.	lv

4.5	Updated package diagram with new suggested entities highlighted in green.	lviii
4.6	Updated deployment diagram with new suggested entities highlighted in green.	lxi

List of Tables

1	Table Example	vi
---	-------------------------	----

Revision History

(Clause 9.2.1)

You can use a table to show your reports' versions and their date.

To create tables, you can use <https://www.latex-tables.com/>; the Table 1 is generated by it.

A	B	C	D

Table 1: Table Example

1. Introduction

This document is prepared to provide the Software Architecture Description (SAD) for the open-source project Koster Seafloor Observatory (KSO), a system designed to support marine biodiversity monitoring by analyzing underwater footage using machine learning and citizen science contributions. The project is a collaboration between researchers, software engineers, and volunteer annotators, currently supported and maintained through open academic and scientific communities. The software architecture presented in this document is based on the requirements identified in the SRS and insights from real-world usage documented by domain experts.

1.1 Purpose and objectives of KSO

The purpose of the Koster Seafloor Observatory (KSO) is to facilitate the ingestion, management, annotation, analysis, and interpretation of underwater footage collected from marine environments—particularly in Sweden’s Kosterhavets National Park. The system enables researchers to monitor marine ecosystems over time and supports large-scale biodiversity studies by integrating automated object detection with human-in-the-loop annotation processes.

Objectives of the KSO system can be summarized as follows:

- Enabling the upload, processing, and storage of underwater video data and associated metadata (e.g., location, depth, time).
- Providing an interactive platform (Zooniverse) where citizen scientists contribute annotations to assist in species identification.

- Training and deploying machine learning models (e.g., YOLOv5/YOLOv8) to automate the classification of marine species.
- Facilitating ecological analysis and reporting based on annotated and predicted data.
- Offering an accessible interface (via Streamlit) for researchers to view, filter, and download results.
- Ensuring interoperability with external biodiversity platforms through standardized APIs and formats.

1.2 Scope

KSO is a multi-component, web-based and notebook-driven software platform aimed at managing and analyzing subsea movie data. It addresses challenges commonly faced in marine research, including high data volume, manual annotation burden, and lack of accessible automation tools for species identification.

The system consists of three primary modules:

- **Data Management:** Handles ingestion of raw footage, cold/hot storage, meta-data association, and retrieval of processed content.
- **Citizen Science Integration:** Enables annotation of species and habitats through Zooniverse; supports consensus calculations and annotation history tracking.
- **Machine Learning:** Provides tools for training object detection models, evaluating model performance, and serving predictions through an API with real-time visualization.

The system is primarily designed for offline ecological analysis, but future extensions may support real-time detection and alerting. It supports deployment on Google Colab, local workstations, or high-performance computing (HPC) clusters. The architecture emphasizes modularity, scalability, and accessibility.

1.3 Stakeholders and their concerns

The system has several stakeholders with varying levels of technical expertise and system interaction roles. These stakeholders and their concerns are summarized as follows:

- **Citizen Scientists:** These users contribute to the annotation process via Zooniverse. They range from enthusiastic volunteers to students and retirees with minimal technical or domain expertise. Their main concern is to interact with an intuitive interface and perform meaningful tasks without requiring specialized training.
- **Researchers and Marine Biologists:** These users rely on KSO for scientific analysis, model evaluation, and biodiversity monitoring. They expect reliable storage, accurate species detection, and easy access to visual reports. They use Streamlit interfaces and Jupyter Notebooks to run experiments and extract ecological insights.
- **Data Scientists and Developers:** These technically skilled users are responsible for enhancing the machine learning pipeline, integrating APIs, and maintaining software modules. Their concerns include reproducibility, maintainability, performance optimization, and extensibility through modular architecture.
- **System Administrators:** These users ensure secure access, manage deployments, and oversee backend operations. They are responsible for managing platform credentials, system configurations, and ensuring compliance with data protection standards such as GDPR.
- **Project Coordinators:** These individuals manage stakeholder coordination, system updates, and strategic planning. Their concerns include usability metrics, annotation progress, scientific output, and community engagement.
- **External Organizations and Conservation Agencies:** These are indirect stakeholders such as government bodies and ecological NGOs. Their interest lies

in accessing validated ecological reports, ensuring ethical data use, and potentially using system insights for conservation policies.

By addressing the varied concerns of these stakeholder groups, the KSO software system ensures usability, accessibility, scientific reliability, and long-term sustainability in marine ecosystem research.

.

2. Glossary

Term/Abbreviation	Description
API	Application Programming Interface. A set of rules that allows one software application to interact with another. In KSO, APIs facilitate communication between system components and external platforms.
CRUD	Create, Read, Update, Delete. Basic operations for persistent storage that are implemented across the KSO system’s data management components.
Darwin Core	A biodiversity data standard that provides a stable, straightforward reference for sharing information about biological diversity. KSO exports data in this format for interoperability with global repositories.
GBIF	Global Biodiversity Information Facility. An international network and data infrastructure funded by governments that provides open access to biodiversity data. KSO integrates with GBIF for data exchange.
HPC	High-Performance Computing. Computing resources with enhanced processing capabilities used by KSO for intensive tasks like ML model training.

Term/Abbreviation	Description
IoU	Intersection over Union. A metric used in object detection to evaluate the accuracy of bounding boxes, especially in the ML component of KSO for species detection.
JWT	JSON Web Token. A compact, URL-safe means of representing claims between two parties, used in KSO for tracking user sessions and authentication.
KSO	Koster Seafloor Observatory. The main system described in this document, designed for marine biodiversity monitoring through video analysis and citizen science.
ML	Machine Learning. A branch of artificial intelligence focused on building systems that learn from data. KSO uses ML for automated species detection and classification.
NGO	Non-Governmental Organization. Private organizations that pursue activities to relieve suffering, promote the interests of the poor, protect the environment, or undertake community development. Some are external stakeholders for KSO.
OBIS	Ocean Biodiversity Information System. A global open-access data and information clearing-house on marine biodiversity that KSO integrates with for data exchange.
REST	Representational State Transfer. An architectural style for designing networked applications, used in KSO's FastAPI interface.
S3	Simple Storage Service. A scalable object storage service used in KSO's backup and data storage strategy.
SAD	Software Architecture Description. This document, describing the architectural structure of the KSO system.
SQLite	A C-language library that implements a small, fast, self-contained SQL database engine. Used by KSO for metadata and annotation storage.

Term/Abbreviation	Description
SRS	Software Requirements Specification. A document that describes the requirements of the KSO system.
Streamlit	An open-source app framework for creating and sharing web apps, used in KSO to provide the researcher interface.
YOLO	You Only Look Once. A family of object detection algorithms used in KSO's machine learning subsystem (YOLOv5/YOLOv8) for automated species identification.
ZFS	Z File System. A combined file system and logical volume manager known for high storage capacity and data integrity features, used in KSO's backup strategy.
Zooniverse	A web-based platform for citizen science projects. Integrated with KSO to enable volunteers to contribute to species annotation tasks.
3-2-1 Rule	A backup strategy that requires keeping three copies of data, stored on two different types of media, with one copy stored off-site. Implemented in KSO's data backup workflow.

3. Architectural Views

3.1 Functional View

3.1.1 Stakeholders' Uses of This View

As defined in Section 1.3 of this document, the Koster Seafloor Observatory (KSO) system has six main stakeholder groups: citizen scientists, researchers and marine biologists, data scientists and developers, system administrators, project coordinators, and external organizations or conservation agencies. Each stakeholder utilizes the Functional View differently based on their roles and responsibilities within the system. Their specific uses of this view are described as follows:

- **Developers and Data Scientists:** This group uses the Functional View as a high-level architectural blueprint to understand the interactions between system components. It provides guidance for implementing modules, integrating APIs, and developing machine learning pipelines. Test engineers also benefit from the Functional View to derive comprehensive testing strategies for system features.
- **Citizen Scientists:** Although not technically focused, citizen scientists may consult the Functional View to gain a basic understanding of how their contributions (e.g., species annotations) are processed within the system and used to improve the AI model and scientific results.
- **Researchers and Marine Biologists:** This stakeholder group refers to the Functional View to understand the lifecycle of data—from footage upload to an-

notation, model training, and results retrieval. This helps them plan experiments, submit data appropriately, and interpret automated outcomes.

- **System Administrators:** Admins use the Functional View to monitor and maintain component interoperability, user access control, data integrity, and system uptime. The view also helps assess the operational impact of configuration or deployment decisions.
- **Project Coordinators:** They use the Functional View to oversee how different user roles and modules work together. This enables them to align stakeholder contributions with project goals and ensure seamless collaboration among technical and non-technical users.
- **External Organizations and Conservation Agencies:** These indirect stakeholders may refer to the Functional View during audits or collaborations to understand how the system generates ecological reports, handles sensitive data, and provides species-related alerts. It also supports transparency and fosters trust in system outcomes.

3.1.2 Component Diagram

The Koster Seafloor Observatory (KSO) system is structured into six main subsystems, each fulfilling distinct functional responsibilities. The system follows a modular architecture that supports scalability, maintainability, and collaboration across multidisciplinary user groups. Figure 3.1 illustrates the relationships and provided/required interfaces among these components.

Component Descriptions:

- **Researcher Subsystem:** Provides the web interface for researchers to upload footage, trigger predictions, and view results. Interfaces include `UploadFootage()`, `StoreRawData()`, `ShareAnnotations()` and `SharePredictionResults()`.

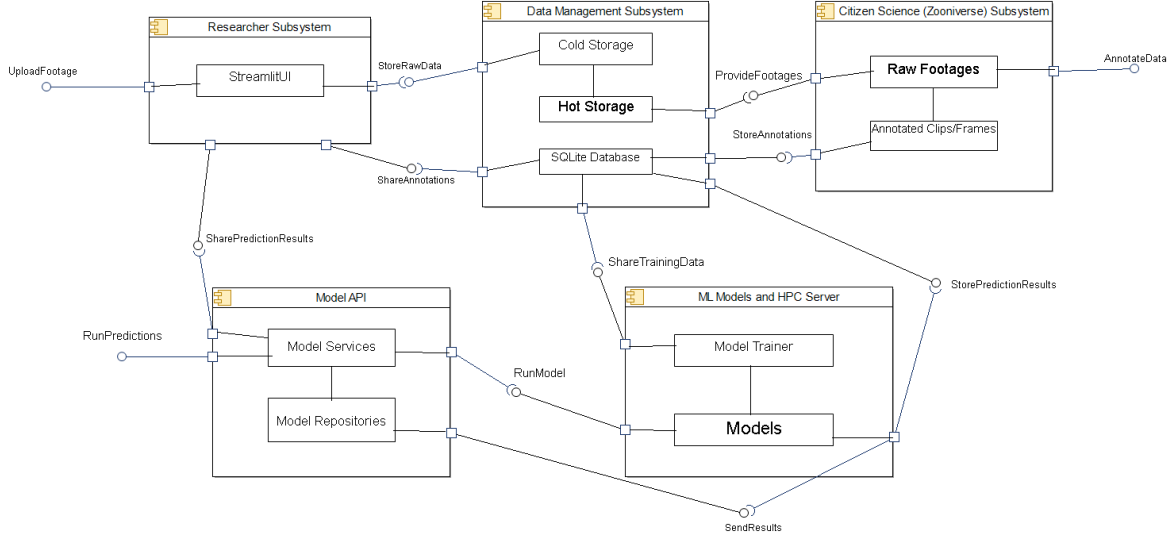


Figure 3.1: Component diagram designed for KSO system.

- **Data Management Subsystem:** Manages cold and hot storage for videos and clips, and stores metadata and annotations in an SQLite database. Key operations include `StoreRawData()`, `ProvideFootages()`, `ShareAnnotations()`, and `ShareTrainingData()`.
- **Citizen Science (Zooniverse) Subsystem:** Receives processed clips for annotation by volunteers. Annotated clips and frames are returned to the data management system. Exposes the `AnnotateData()` interface.
- **ML Models and HPC Server:** Uses annotated data to train machine learning models for species detection (e.g., YOLO). Supports model training and prediction via `RunModel()`, `StorePredictionResults()`, and `SendResults()`.
- **Model API:** Serves as the public-facing gateway for running predictions and retrieving model results. Interfaces include `RunPredictions()` and `SharePredictionResults()`.

This architecture supports collaborative workflows between researchers, volunteers, and ML developers by clearly separating concerns. Each component interacts through well-defined interfaces, making the system easier to test, extend, and deploy.

3.1.3 Internal Interfaces

This section presents the core internal interfaces of the Koster Seafloor Observatory (KSO) system through a class diagram and detailed component descriptions. Figure 3.2 illustrates the four key interfaces and their interactions.

Component Descriptions

1. Annotation Data Service

- **Purpose:** Manages labeled marine biodiversity data from both machine learning and human annotators
- **Attributes:**
 - `datasetID`: Unique identifier for annotation sets
 - `annotations`: JSON array of labeled bounding boxes and species metadata
 - `lastUpdated`: Timestamp of most recent modification
- **Operations:**
 - `submitAnnotation(userID, labels)`: Validates and stores citizen scientist inputs
 - `getAnnotations(datasetID)`: Retrieves annotation sets
 - `validateAnnotations(mlResults)`: Compares human labels with ML predictions
 - `exportToBiodiversityAPI()`: Formats data to Darwin Core standards

2. ML Inference API

- **Purpose:** Handles automated species detection and model improvement
- **Attributes:**
 - `modelVersion`: Active YOLOv5/ResNet model version

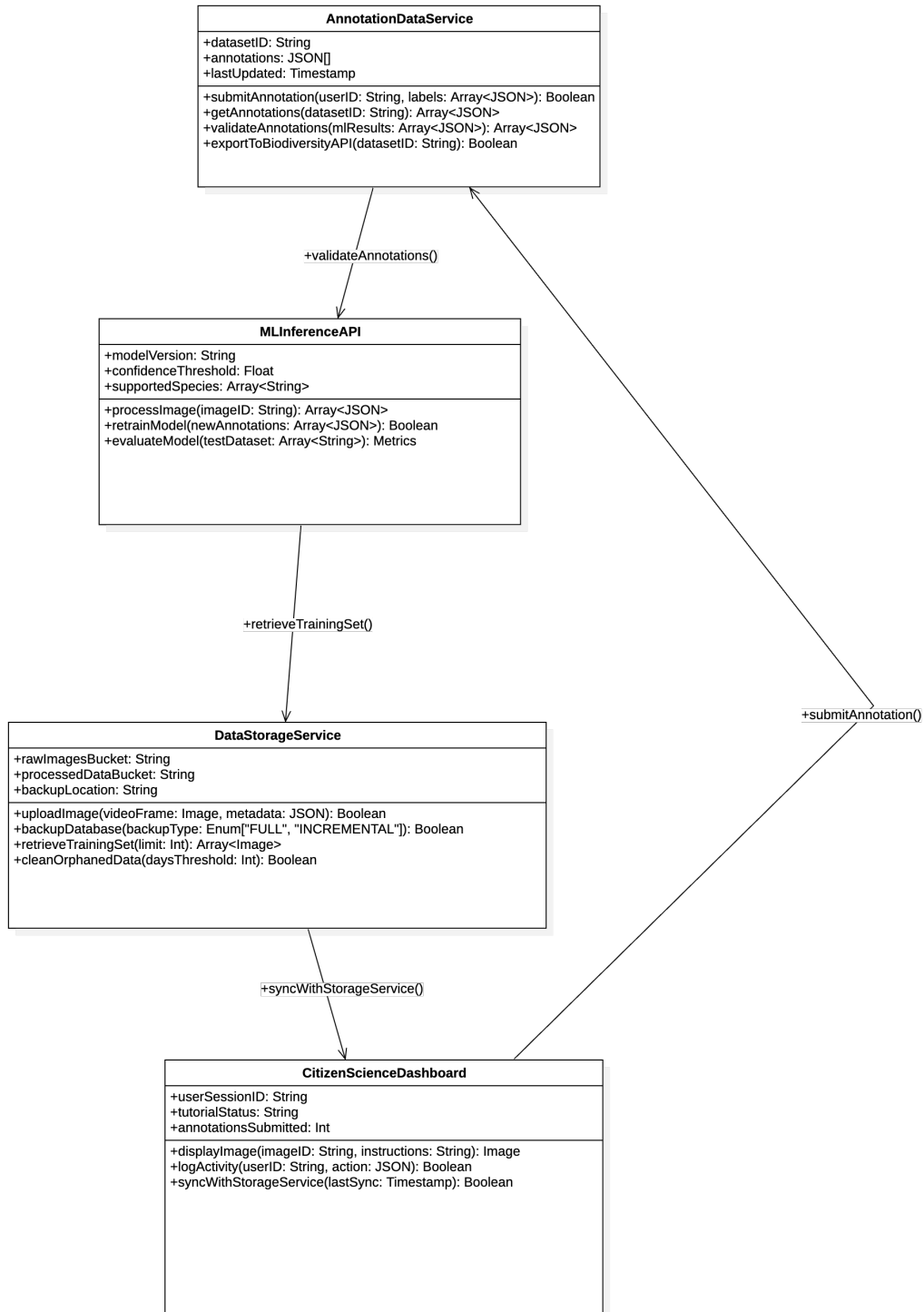


Figure 3.2: Internal Interfaces Class Diagram of the KSO System

- `confidenceThreshold`: Minimum score (0.7 default) for species detection

- **Operations:**

- `processImage(imageID)`: Executes inference on new images
- `retrainModel(newAnnotations)`: Initiates transfer learning
- `evaluateModel(testDataset)`: Generates performance metrics

3. Citizen Science Dashboard

- **Purpose:** Web interface for volunteer participation

- **Attributes:**

- `userSessionID`: JWT token for activity tracking
- `tutorialStatus`: User onboarding progress tracker

- **Operations:**

- `displayImage(imageID)`: Serves images with annotation tools
- `logActivity(userID, action)`: Records user actions
- `syncWithStorageService()`: Maintains image cache

4. Data Storage Service

- **Purpose:** Manages distributed data with redundancy

- **Attributes:**

- `rawImagesBucket`: S3-compatible storage for original frames
- `processedDataBucket`: Encrypted storage for ML outputs

- **Operations:**

- `uploadImage(videoFrame)`: Ingests frames with metadata

- `retrieveTrainingSet()`: Provides balanced datasets
- `cleanOrphanedData()`: Removes unreferenced files

Interaction Workflows

- **Annotation Validation:**

1. User submits labels via Dashboard
2. Annotation Service stores entries as `pendingValidation`
3. ML API validates labels hourly
4. Confirmed annotations marked `verified`

- **Model Retraining:**

1. Triggered at 1000+ verified annotations
2. Storage Service prepares dataset
3. ML API executes retraining
4. New model deployed after evaluation

3.1.4 Interaction Patterns

This section includes **3 Sequence Diagrams** to show messaging sequences taking place among the system components over the **internal interfaces**. **Choose the 3 most complex interactions to model with sequence diagrams.**

The Koster Seafloor Observatory (KSO) system relies on carefully orchestrated interactions between its internal components. This section details three critical messaging sequences that ensure robust operation of the system. Each interaction is represented as a sequence diagram and explained in detail below.

1. Citizen Science Annotation Validation

Purpose: Ensures user-submitted annotations align with machine learning predictions to maintain data quality.

Key Steps:

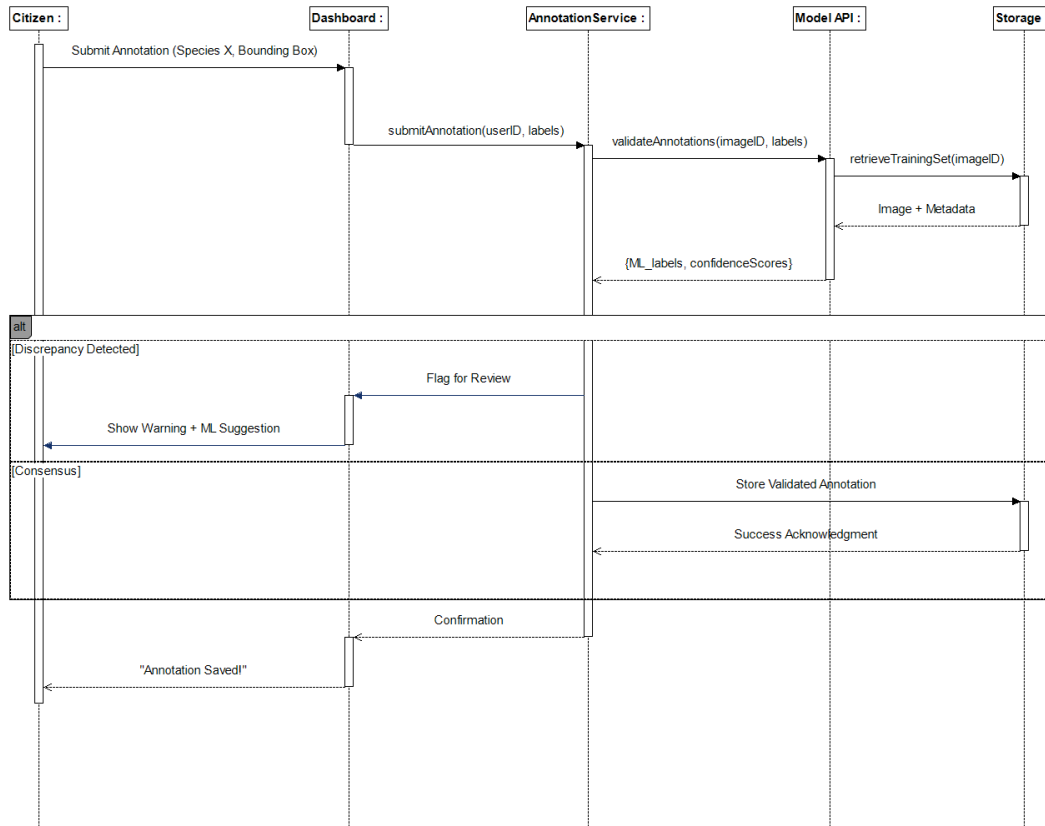


Figure 3.3: Sequence diagram showing the annotation validation process between citizen scientists, the dashboard, and backend services.

1. The citizen scientist submits an annotation (species label and bounding box) through the Dashboard interface.
2. The Dashboard forwards the annotation to the Annotation Service via the `submitAnnotation()` method.
3. The Annotation Service requests validation from the ML Inference API, providing the image ID and user labels.
4. The ML API retrieves the original image from Storage for reprocessing.
5. The ML API compares its predictions with the user's annotations:
 - If discrepancies exist (low confidence), the system flags the annotation for review.
 - If consensus is reached, the annotation is stored in the database.
6. The Dashboard notifies the user of the result.

Error Handling: Failed validations trigger an automatic review process without blocking the user.

2. Model Retraining Pipeline

Purpose: Periodically improves machine learning models using newly verified annotations.

Key Steps:

1. A scheduled job triggers the retraining process weekly.
2. The Annotation Service retrieves all verified annotations from Storage.
3. The ML API fetches a balanced training set of images and labels.
4. The system trains a new model version while evaluating performance metrics.
5. If metrics improve (e.g., F1 score $\geq$ threshold), the new model is deployed.
6. The old model is archived with version control for potential rollback.

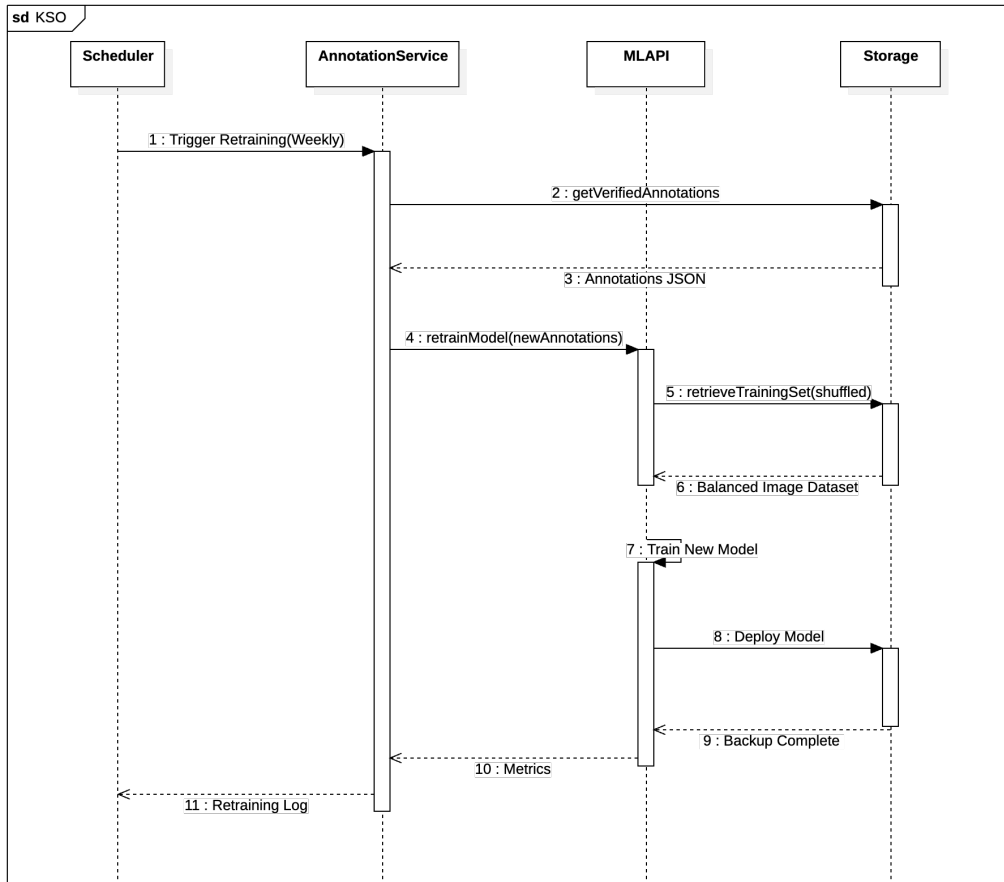


Figure 3.4: Sequence diagram of the automated model retraining workflow.

Performance Consideration: Training occurs offline to avoid impacting real-time user operations.

3. Data Backup and Recovery

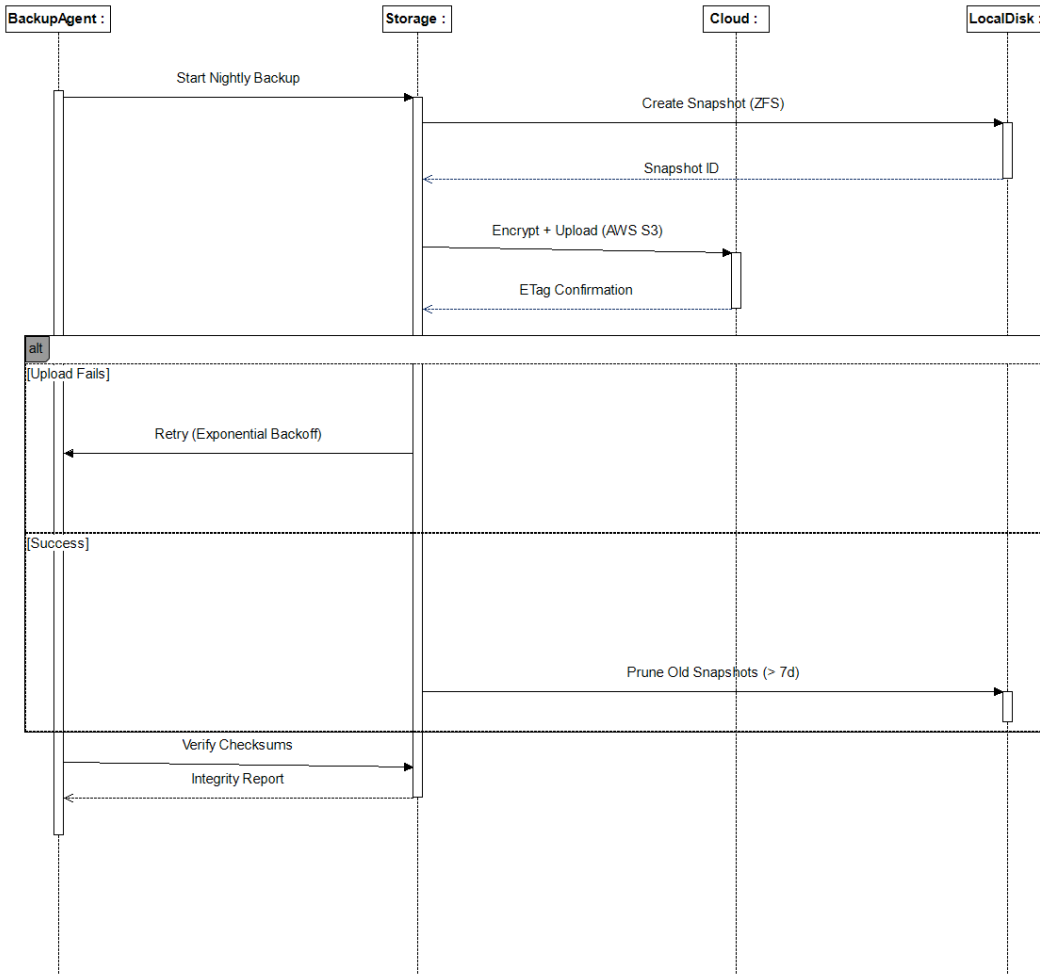


Figure 3.5: Sequence diagram of the distributed backup process following the 3-2-1 rule.

Purpose: Implements a fault-tolerant storage strategy for critical data.

Key Steps:

1. The Backup Agent initiates nightly backups.
2. Storage creates a ZFS snapshot on local disks.
3. Data is encrypted and uploaded to cloud storage (AWS S3).

4. The system handles failures with exponential backoff retries.
5. Old snapshots (7 days) are automatically pruned.
6. Checksum verification ensures data integrity post-backup.

Safety Features:

- 3-2-1 backup rule implementation
- Atomic snapshot operations prevent partial backups
- End-to-end encryption for cloud transfers

Design Rationale The selected interaction patterns represent the most complex and critical workflows in KSO:

- Annotation validation combines human and machine inputs
- Model retraining demonstrates offline batch processing
- Backup procedures ensure system reliability

3.1.5 External Interfaces

This section includes the **External Interfaces Class Diagram** and descriptions of the external interfaces used in the KSO system.

1. Streamlit Researcher Interface

This interface is designed for researchers to interact with the system. It enables uploading media, setting model parameters, visualizing results, and exporting annotations.

- `upload_media(file: byte): string` – Uploads a video file and returns a status string or file identifier.
- `set_detection_params(conf: float, iou: float)` – Sets the confidence and IoU thresholds for the detection model.

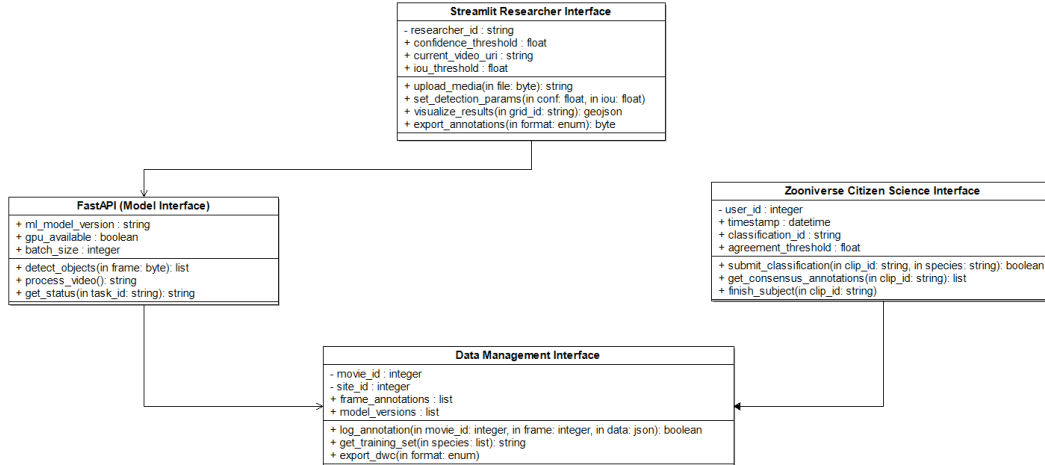


Figure 3.6: External interfaces diagram of KSO system.

- `visualize_results(grid_id: string): geojson` – Displays detection results for a given area as GeoJSON.
- `export_annotations(format: enum): byte` – Exports annotations in the desired file format.

2. FastAPI (Model Interface)

This RESTful interface connects the Streamlit frontend to backend ML models. It handles detection tasks and provides model-related metadata.

- `detect_objects(frame: byte): list` – Processes a single frame and returns a list of detected objects.
- `process_video(): string` – Triggers asynchronous video processing, returning a task ID.
- `get_status(task_id: string): string` – Retrieves the processing status of a video task.

3. Zooniverse Citizen Science Interface

This interface allows citizen scientists to participate in the classification process, submitting observations and receiving feedback on consensus.

- `submit_classification(clip_id: string, species: string): boolean` – Records a classification for a specific clip.
- `get_consensus_annotations(clip_id: string): list` – Retrieves agreed-upon annotations from multiple users.
- `finish_subject(clip_id: string)` – Marks the classification process for a clip as complete.

4. Data Management Interface

This interface stores and manages annotated data and model versions, supports training set generation, and enables data export.

- `log_annotation(movie_id: int, frame: int, data: json): boolean` – Saves annotations for a specific video frame.
- `get_training_set(species: list): string` – Extracts training data for specified species.
- `export_dwc(format: enum)` – Exports dataset in Darwin Core or other specified formats.

3.1.6 Interaction Scenarios

This section presents **two Activity Diagrams** that model complex interaction sequences involving external interfaces within the Koster Seafloor Observatory (KSO) system. These diagrams depict how different actors (such as researchers and citizen scientists) interact with system components through external interfaces including the Streamlit-based researcher dashboard, the FastAPI model service, the Zooniverse platform, and the internal data management interface.

The interaction scenarios modeled here are selected based on their architectural relevance and dynamic complexity. They provide additional insights into the behavioral view of the system's runtime logic.

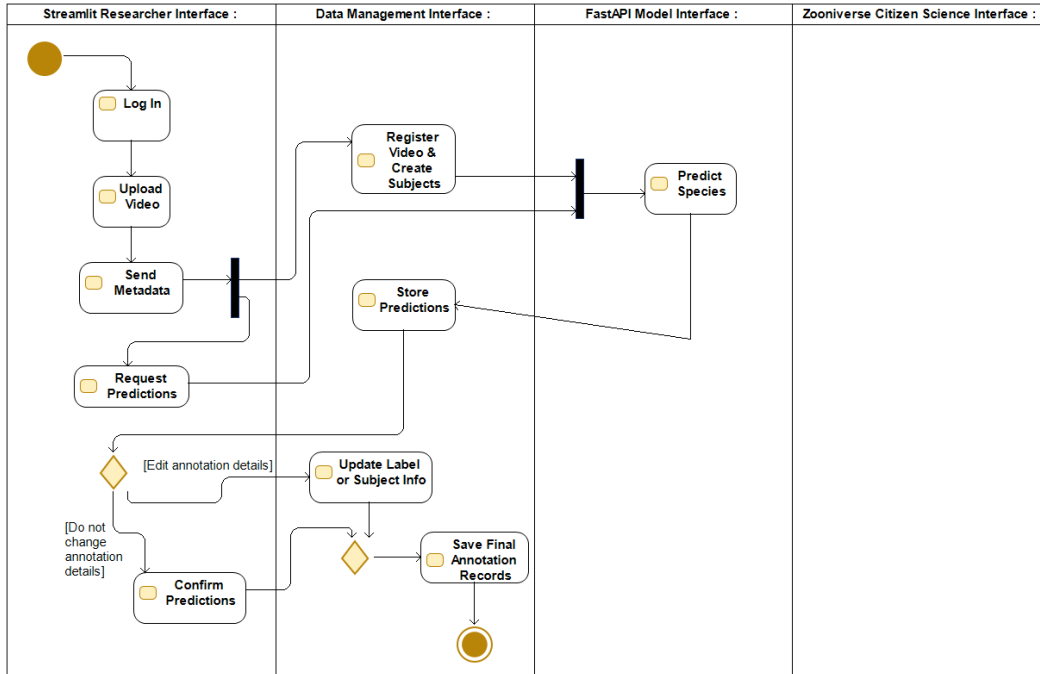


Figure 3.7: Activity Diagram of Researcher-Guided Automated Annotation Workflow.

The first diagram (Figure 3.7) illustrates the process in which a researcher uses the Streamlit interface to upload subsea videos, initiate machine learning predictions through FastAPI, and optionally update or confirm predicted annotations. This workflow emphasizes the integration of automated analysis with human-in-the-loop validation, supported by both front-end and back-end system components.

The second diagram (Figure 3.8) details the interaction between citizen scientists and the system via the Zooniverse platform. It includes decision points based on annotator confidence and backend processes for storing, aggregating, and validating the incoming annotations. This scenario highlights how public participation contributes to reliable scientific data through iterative consensus mechanisms.

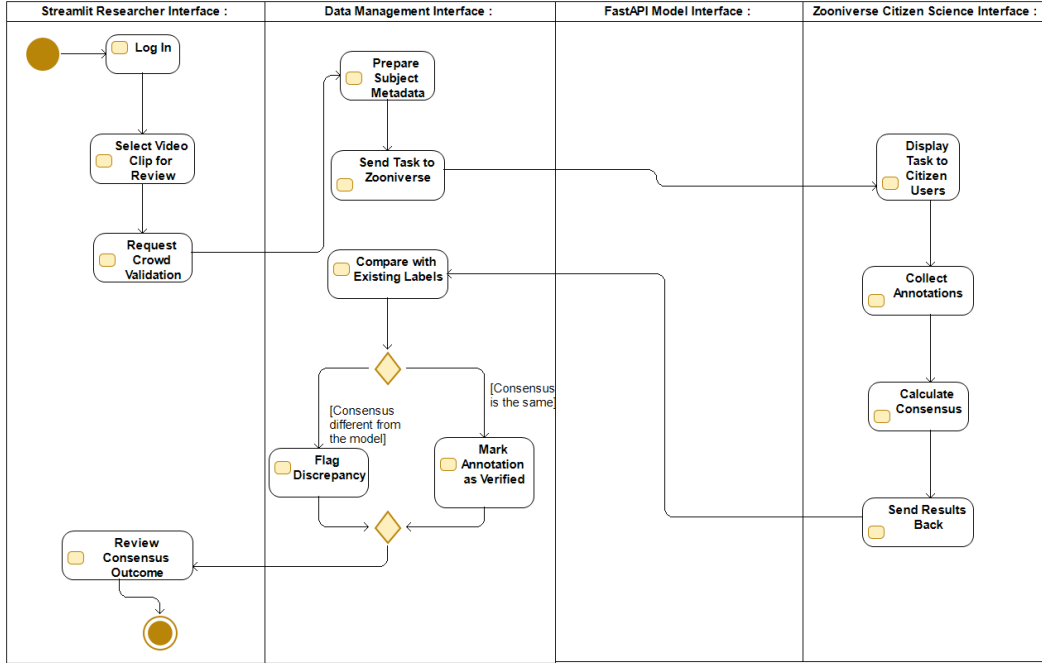


Figure 3.8: Activity Diagram of Crowdsourced Annotation Validation Scenario.

3.2 Information View

3.2.1 Stakeholders' Uses of This View

The information view captures the structure, organization, and flow of data in the KSO system. It includes key entities such as movies, subjects, species, annotations, and user roles. Each stakeholder group interacts with this view in different ways to fulfill their responsibilities and objectives within the system.

- **Citizen Scientists:** Utilize the system to annotate and classify subsea video clips. The information view reflects how their contributions are stored, linked to video subjects, and later aggregated. This understanding is essential for improving engagement mechanisms and tracking contribution impact.
- **Researchers and Marine Biologists:** Rely on accurate structuring of observational and annotated data. The information view helps them trace annotations to video content, review species metadata, and validate the outputs of citizen

science and machine learning processes.

- **Data Scientists and Developers:** Use the view to design, maintain, and improve data pipelines, APIs, and analytical models. Understanding the data schema is critical for integrating machine learning predictions, optimizing queries, and maintaining data integrity across components.
- **System Administrators:** Leverage the view to manage user roles, control access, enforce data consistency, and monitor system-level data operations. It supports database maintenance, backup strategies, and user auditing.
- **Project Coordinators:** Use the information view to ensure that all data-related workflows align with project objectives. They monitor data collection progress, annotation coverage, and stakeholder engagement using insights derived from structured data.
- **External Organizations or Conservation Agencies:** Access aggregated data outputs to support conservation policy-making and environmental research. The information view helps them understand data provenance, quality, and relationships between ecological observations and outcomes.

3.2.2 Database Class Diagram

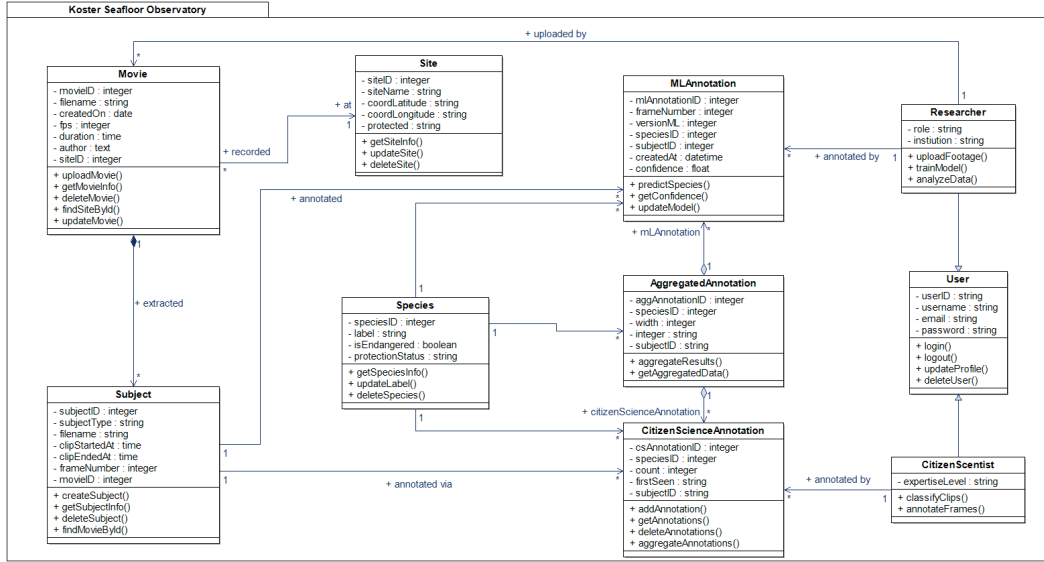


Figure 3.9: Database class diagram of KSO system.

3.2.3 Operations on Data

Descriptions of the operations are given in the database class diagram. These operations deal with the storage and handling of information regarding the application domain entities such as movies, subjects, species, annotations, and users. Most of these operations fall under the CRUD (Create, Read, Update, Delete) pattern.

- `uploadMovie()` – Insert a new movie record. (*Movie*)
- `getMovieInfo()` – Retrieve movie metadata. (*Movie*)
- `deleteMovie()` – Remove a movie record. (*Movie*)
- `findSiteById()` – Fetch the site related to a movie. (*Movie*)
- `updateMovie()` – Update existing movie information. (*Movie*)
- `createSubject()` – Create a new subject extracted from a movie. (*Subject*)

- `getSubjectInfo()` – Retrieve subject metadata. (*Subject*)
- `deleteSubject()` – Remove a subject record. (*Subject*)
- `findMovieById()` – Find the associated movie for a subject. (*Subject*)
- `getSiteInfo()` – Retrieve information about a site. (*Site*)
- `updateSite()` – Update site metadata. (*Site*)
- `deleteSite()` – Delete an observation site. (*Site*)
- `getSpeciesInfo()` – Retrieve biological and conservation info. (*Species*)
- `updateLabel()` – Update species classification label. (*Species*)
- `deleteSpecies()` – Delete a species entry. (*Species*)
- `predictSpecies()` – Predict species using machine learning. (*MLAnnotation*)
- `getConfidence()` – Retrieve prediction confidence. (*MLAnnotation*)
- `updateModel()` – Update or retrain the ML model. (*MLAnnotation*)
- `aggregateResults()` – Aggregate annotations across methods. (*AggregateAnnotation*)
- `getAggregatedData()` – Retrieve the combined annotation results. (*AggregateAnnotation*)
- `addAnnotation()` – Submit a citizen science annotation. (*CitizenScienceAnnotation*)
- `getAnnotations()` – Retrieve annotations submitted by users. (*CitizenScienceAnnotation*)
- `deleteAnnotations()` – Remove user-submitted annotations. (*CitizenScienceAnnotation*)

- `aggregateAnnotations()` – Aggregate citizen contributions. (*CitizenScienceAnnotation*)
- `uploadFootage()` – Upload video content for analysis. (*Researcher*)
- `trainModel()` – Train ML model using collected data. (*Researcher*)
- `analyzeData()` – Analyze video content using models. (*Researcher*)
- `classifyClips()` – Classify video clips by observing them. (*CitizenScientist*)
- `annotateFrames()` – Annotate individual video frames. (*CitizenScientist*)
- `login()` – Authenticate into the system. (*User*)
- `logout()` – End a user session. (*User*)
- `updateProfile()` – Modify user profile details. (*User*)
- `deleteUser()` – Permanently remove a user account. (*User*)

3.3 Development View

3.3.1 Stakeholders’ Uses of This View

The development (package) view organises source code into cohesive modules, clarifies compilation dependencies, and reveals extension points. Each stakeholder group consults this view in different ways:

- **Core Developers and Feature Teams:** Locate where to add or refactor code (for example, a new model inside `MachineLearning::ModelTraining`) without touching unrelated packages.
- **QA and Test Engineers:** Use package boundaries to design unit and integration tests, focusing on high-risk calls such as `AnnotationSystem` to `MachineLearning`.

- **Build and Release Engineers:** Configure CI/CD pipelines, parallelise builds based on independent packages, and trigger selective deployments when only some modules change.
- **Security Reviewers:** Pin-point where to run static analysis, licence checks, or secret scanning—especially in code that handles credentials or external APIs.
- **New Contributors and Open-Source Community:** Use the high-level package map as an entry guide to find the right folder for bug fixes or plug-in integrations.
- **Project Coordinators:** Track work allocation by mapping user stories to packages and spotting hotspots where multiple teams converge.
- **Technical Writers and API Documenters:** Align documentation sections with package names so readers can navigate code and docs side by side.
- **DevOps and Site-Reliability Engineers:** Correlate runtime failures back to the owning code package, streamlining incident response and on-call hand-offs.

3.3.2 Package Diagram

This section should include a **Package Diagram** showing the higher-level software modules and their dependencies. Package decomposition hierarchy must involve 3 levels, namely, (level 0) the system (kso), (level 1) subsystems of kso, and (level 2) sub-subsystem levels. **Briefly explain each package’s contents and show some dependencies between the packages.**

The diagram illustrates the modular architecture of the KSO (Knowledge-based Species Observation) system. It is hierarchically structured to depict the dependencies and relationships among various functional packages. Higher-level modules rely on lower-level ones, with the vertical arrangement visually emphasizing this layered dependency.

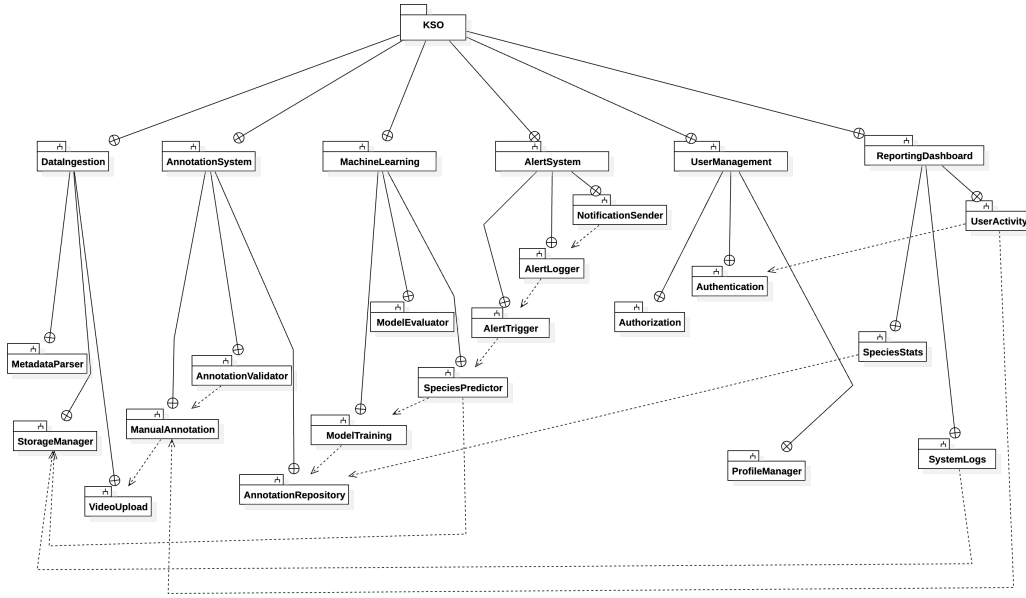


Figure 3.10: Package diagram of the KSO System.

Subsystem Overview

1. DataIngestion Purpose: Manages the pipeline for ingesting raw data, such as video footage and associated metadata.

- **VideoUpload** — Handles video ingestion and validates uploaded files.
- **MetadataParser** — Extracts and formats metadata from uploaded content.
- **StorageManager** — Stores uploaded files and ensures accessibility for other modules.

2. AnnotationSystem Purpose: Supports manual annotation of ingested video content.

- **ManualAnnotation** — Provides tools for human annotators to label content.
- **AnnotationRepository** — Centralized storage for annotation data.
- **AnnotationValidator** — Ensures the quality and consistency of annotations.

3. MachineLearning Purpose: Hosts components for model training, evaluation, and inference.

- **ModelTraining** — Trains machine learning models using labeled data.
- **ModelEvaluator** — Assesses model performance.
- **SpeciesPredictor** — Infers species from input data.

4. AlertSystem Purpose: Monitors predictions and triggers alerts when specific conditions are met.

- **AlertTrigger** — Defines and evaluates alert conditions.
- **AlertLogger** — Logs triggered alert events.
- **NotificationSender** — Dispatches alerts through suitable channels.

5. UserManagement Purpose: Manages user identities and access permissions.

- **Authentication** — Verifies user credentials.
- **Authorization** — Controls access based on user roles.
- **ProfileManager** — Manages user profile data and preferences.

6. ReportingDashboard Purpose: Delivers visual insights and activity monitoring for the system.

- **UserActivity** — Tracks interactions performed by users.
- **SpeciesStats** — Aggregates and visualizes species-related predictions.
- **SystemLogs** — Records system-level events for diagnostics.

Inter-Package Dependencies

Source Package	Target Package	Dependency Description
ManualAnnotation	VideoUpload	Uses video data for annotation.
AnnotationRepository	ManualAnnotation	Stores annotations created during manual labeling.
AnnotationValidator	AnnotationRepository	Validates annotations for consistency and accuracy.
ModelTraining	AnnotationRepository	Uses labeled data for training ML models.
SpeciesPredictor	ModelTraining	Performs inference using trained models.
AlertTrigger	SpeciesPredictor	Triggers alerts based on predictions.
AlertTrigger	AlertLogger	Logs alerts once triggered.
AlertLogger	NotificationSender	Sends notifications following event logging.
ProfileManager	Authentication	Relies on user identity verification.
Authorization	Authentication	Grants access post-authentication.
UserActivity	ManualAnnotation	Tracks annotation-related user actions.
UserActivity	ProfileManager	Associates activities with user profiles.
SpeciesStats	SpeciesPredictor	Visualizes species detection results.
SystemLogs	NotificationSender	Records alert messages.
SystemLogs	StorageManager	Logs events related to storage.

Hierarchical Structure

- **Level 0 (Top Level):** *KSO* — The system orchestrator overseeing all modules.
- **Level 1 (Subsystems):** Core functional packages — *DataIngestion*, *AnnotationSystem*, *MachineLearning*, *AlertSystem*, *UserManagement*, *ReportingDashboard*.
- **Level 2 (Components):** Internal modules within each package — e.g., *VideoUpload*, *MetadataParser*, *SpeciesPredictor*.

3.4 Deployment View

3.4.1 Stakeholders' Uses of This View

The deployment view shows how software artifacts are mapped onto physical or virtual infrastructure, highlighting network boundaries, resource constraints, and failure domains. Different stakeholder groups consult this view for the purposes listed below.

- **System Administrators and DevOps Engineers:** Size compute clusters, set up auto-scaling, configure monitoring, and plan backup or disaster-recovery regions based on the node layout shown in the deployment diagram.
- **Developers and Data Scientists:** See which runtime environments (GPU nodes, message queues, object storage) their services will run on, so they can build containers and debug jobs locally before deployment.
- **Security and Compliance Officers:** Verify network zones, firewall boundaries, and data-at-rest locations to make sure the platform meets GDPR and institutional security policies.
- **Project Coordinators and Budget Owners:** Estimate cloud cost drivers such as GPU hours, storage growth, and egress bandwidth, making informed decisions on reserved instances or on-prem alternatives.
- **Researchers and Marine Biologists:** Confirm that raw footage remains in encrypted storage and that compute-intensive retraining workloads do not impede daytime access to dashboards.
- **External Organizations or Conservation Agencies:** During audits or data-sharing agreements, inspect where data is processed, backed up, and exported to third-party biodiversity hubs.
- **Cloud Infrastructure Providers (indirect):** Use the diagram as an integration checklist when provisioning virtual networks, managed message queues, GPU

fleets, or cross-region replication services for the platform.

3.4.2 Deployment Diagram

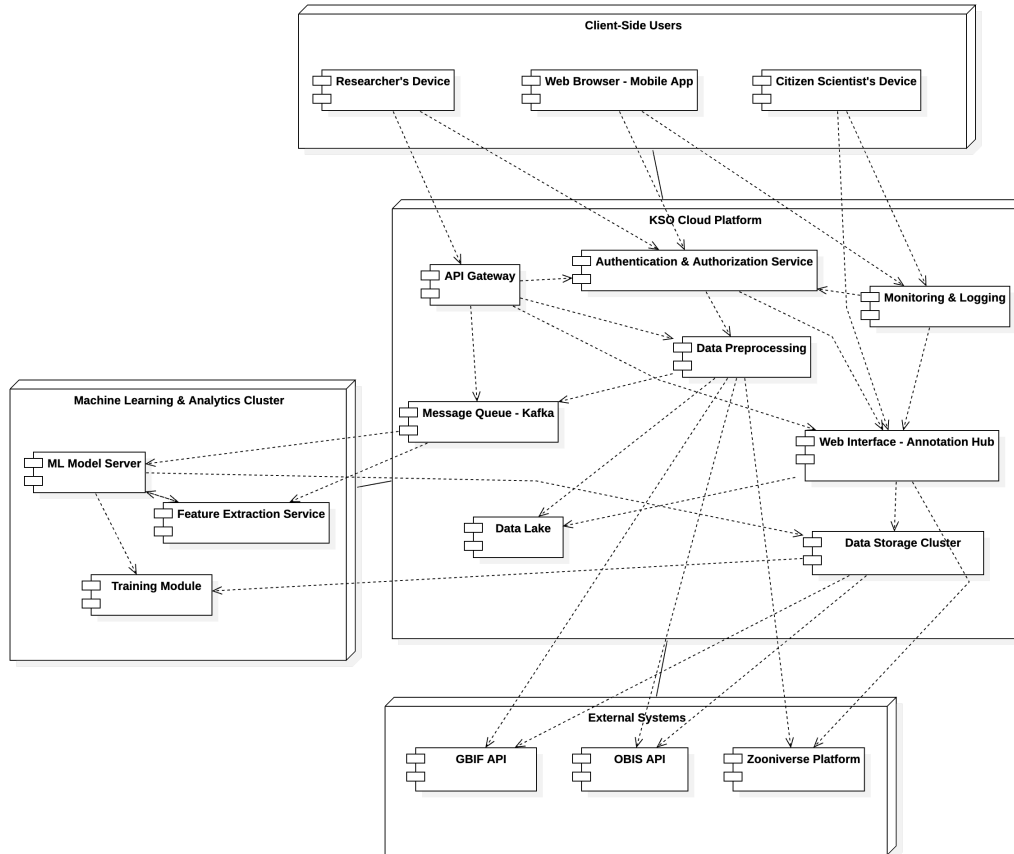


Figure 3.11: Deployment diagram of the KSO System.

This section explains the UML Deployment Diagram of the cloud-based AI/ML platform for biodiversity data analysis. The system involves interactions between client devices, a central cloud platform (KSO Cloud Platform), machine learning clusters, and external data providers (e.g. Zooniverse).

1. Nodes and Their Roles

1.1 Client-Side Users

- **Researcher's Device:** Used by professional scientists to access and analyze data.

- **Web Browser / Mobile App:** Generic frontends for interaction with APIs and dashboards.
- **Citizen Scientist's Device:** Enables public contribution via image tagging or observation.

1.2 KSO Cloud Platform This is the core backend platform and includes the following components:

- **API Gateway:** Routes all external requests.
- **Authentication & Authorization Service:** Validates users and access rights.
- **Data Preprocessing:** Cleans and transforms incoming raw data.
- **Web Interface - Annotation Hub:** Web dashboard for annotations and data visualization.
- **Monitoring & Logging:** Observability tools for tracking metrics and errors.
- **Message Queue (Kafka):** Ensures asynchronous data flow between services.
- **Data Lake:** Stores raw, semi-structured data.
- **Data Storage Cluster:** Stores structured and annotated data.

1.3 Machine Learning & Analytics Cluster

- **ML Model Server:** Hosts machine learning models for inference.
- **Feature Extraction Service:** Derives ML features from raw data.
- **Training Module:** Retrains models with labeled or updated datasets.

1.4 External Systems These are third-party APIs or platforms the system depends on:

- **GBIF API:** Provides biodiversity data.
- **OBIS API:** Supplies marine species records.
- **Zooniverse Platform:** Source of crowd-annotated datasets.

2. Communication Path: Node-to-Node Interactions

From Node	To Node	Explanation
Client-Side Users	KSO Cloud Platform	Devices interact via API Gateway for submitting data or fetching analytics.
Machine Learning & Analytics Cluster	KSO Cloud Platform	Receives training data, returns inference results, and stores processed outputs.
External Systems	KSO Cloud Platform	Biodiversity data flows into preprocessing services via APIs.

3. Dependency: Component-to-Component and External Interactions

From Component	To Component / API	Explanation
API Gateway	Authentication & Authorization	For authenticating incoming user/API requests.
API Gateway	Data Preprocessing	Sends user-submitted data to be cleaned and validated.
API Gateway	Web Interface - Annotation Hub	Serves content and routes data to UI layer.
Authentication & Authorization	Annotation Hub	Allows/denies access to annotation tools.
Authentication & Authorization	Data Preprocessing	Ensures only authenticated data flows to preprocessing.

Data Preprocessing	GBIF API	Fetches raw biodiversity records from GBIF.
Data Preprocessing	OBIS API	Ingests marine biodiversity data.
Data Preprocessing	Zooniverse Platform	Retrieves citizen science labeled datasets.
Data Preprocessing	Kafka (Message Queue)	Publishes cleaned data for async delivery to downstream services.
Data Preprocessing	Data Lake	Stores cleaned or raw data in large, unstructured formats.
Kafka	ML Model Server	Streams data for real-time inference or batch model usage.
Kafka	Feature Extraction Service	Pushes data for transformation into machine learning features.
ML Model Server	Feature Extraction Service	Depends on preprocessed features for accurate predictions.
ML Model Server	Training Module	Sends results or feedback to retrain/update models.
Web Interface - Annotation Hub	Data Storage Cluster	Reads/writes annotations and structured analysis results.

Monitoring & Logging	All KSO Services	Collects operational metrics and error logs across components.
----------------------	------------------	--

4. Component Role Summary

Component	Role in the System
API Gateway	Entry point for all client requests; routes to services.
Authentication & Authorization	Verifies user identity and access levels.
Data Preprocessing	Cleans and transforms raw data from external sources.
Kafka Message Queue	Enables asynchronous communication between components.
Web Interface – Annotation Hub	User interface for data labeling and insights.
Monitoring & Logging	Tracks errors, usage, and system health.
Data Lake	Stores raw and semi-structured incoming data.
Data Storage Cluster	Structured, queryable storage for processed data.
ML Model Server	Hosts trained models for real-time inference.
Feature Extraction Service	Transforms raw inputs into model-friendly features.
Training Module	Retrains ML models with curated data.
GBIF/OBIS APIs	Data sources for biodiversity occurrences.
Zooniverse Platform	Source of citizen-labeled or crowd-annotated data.

This section should include a **Deployment Diagram and explanations** for the deployment diagram to show a possible allocation of components over resources.

3.5 Design Rationale

This section presents the key design rationales for each architectural view of the Koster Seafloor Observatory (KSO) system, highlighting the driving factors and trade-offs considered in our architectural decisions.

3.5.1 Functional View Rationale

The functional view adopts a modular, component-based architecture to support the diverse and evolving needs of the KSO ecosystem. The primary design decision was to separate the system into distinct subsystems with well-defined interfaces, each targeting specific stakeholder needs. This approach enables:

- **Collaborative Development:** By clearly defining interfaces between components like the Researcher Subsystem, Data Management Subsystem, and ML Models, we enable parallel development by distributed teams with different expertise (marine biology, ML, citizen science).
- **Incremental Enhancement:** The modularized approach allows for targeted improvements to individual components (e.g., upgrading ML models) without disrupting the entire system, critical for a research platform that must evolve with scientific advances.
- **Stakeholder-Centric Design:** Each component directly addresses the needs of specific stakeholder groups—researchers interact primarily with their dedicated subsystem, while citizen scientists engage through the Zooniverse interface, ensuring each user group has an optimized experience.

Alternative approaches considered included a more monolithic system and a fully microservice-based architecture. The monolithic approach was rejected due to its limited flexibility for future extensions, while a pure microservice architecture introduced unnecessary complexity for the current scale of operations. The selected component-based architecture balances flexibility with manageable complexity.

3.5.2 Information View Rationale

The information view implements a domain-driven design that prioritizes scientific accuracy and data traceability. The central design decision was to structure data entities around the natural workflow of marine research:

- **Science-Driven Schema:** The database schema mirrors the real-world relationships in marine research—from observation sites to specific species—ensuring that data remains meaningful to researchers while supporting computational needs.
- **Annotation Provenance:** The system maintains clear separation between machine learning annotations, citizen science contributions, and aggregate results, enabling quality assessment and trustworthy scientific outputs through transparent provenance tracking.
- **Integration with Biodiversity Standards:** Entities are designed to facilitate conversion to established standards like Darwin Core, enhancing interoperability with global biodiversity databases like GBIF and OBIS.

We considered simplified schemas that would merge annotation types or flatten the hierarchy, but these approaches would have compromised data lineage tracking and reduced the system’s scientific credibility. The current design optimizes for scientific integrity while maintaining reasonable query performance.

3.5.3 Development View Rationale

The development view employs a three-tier package hierarchy that balances cohesion with manageable dependencies. The key design decisions include:

- **Domain-Focused Packages:** Organizing packages around domain concerns (DataIngestion, AnnotationSystem, MachineLearning) rather than technical layers creates higher cohesion and clearer responsibility boundaries for development teams.

- **Controlled Dependencies:** The hierarchical structure with carefully managed dependencies minimizes coupling between unrelated components, reducing the risk of changes in one area affecting others.
- **Extensibility Points:** Strategic interfaces in packages like the `AlertSystem` and `ReportingDashboard` allow for future extensions without modifying core functionality.

Alternative organizational schemes considered included a purely technical layering (presentation, business logic, data access) and a feature-based approach. The technical layering would have created excessive cross-cutting concerns, while the feature-based approach would have led to code duplication. Our domain-focused approach provides better alignment with team expertise and system evolution patterns.

3.5.4 Deployment View Rationale

The deployment view adopts a cloud-native, distributed architecture that prioritizes scalability and resilience. The principal design decisions were:

- **Separation of Compute-Intensive Workloads:** ML training and inference are deployed on dedicated clusters separate from the main application services, allowing independent scaling of these resource-intensive operations.
- **Asynchronous Processing Pattern:** Implementing Kafka as a message queue between components enables the system to handle variable loads from video processing and annotation activities without performance degradation.
- **Hybrid Storage Strategy:** Using a combination of data lake (for raw footage) and structured storage (for annotations and metadata) optimizes for both cost-effectiveness and query performance.

We evaluated both fully on-premises solutions and serverless architectures as alternatives. The on-premises approach would have limited scalability for the computationally

intensive ML workloads, while a purely serverless design would have complicated long-running training jobs and increased latency for real-time predictions. The hybrid cloud architecture with dedicated ML clusters provides the optimal balance of performance, cost, and operational flexibility for the diverse workloads in the KSO system.

4. Architectural Views for Your Suggestions to Improve the Existing System

4.1 Functional View

4.1.1 Stakeholders' Uses of This View

- **Developers and Data Scientists** use the diagram to see how the new *Active-Learning & Annotation Orchestrator (ALAO)* plugs into the existing data-model loop, letting them implement or tune active-learning logic without touching the model-training code.
- **Citizen Scientists** gain a quick picture of how their annotations are now *actively* requested by TaskScheduler in ALAO and how their verified labels improve future model runs.
- **Researchers and Marine Biologists** follow the full data life-cycle—from footage upload through annotation, active-learning feedback, retraining, and automated results—so they can plan experiments accordingly.
- **System Administrators** understand the new runtime dependencies (e.g. ALAO $\leftrightarrow$ ML Server, ALAO $\leftrightarrow$ Zooniverse) when configuring messaging, access control, and monitoring.

- **Project Coordinators** confirm that technical and non-technical contributions align, now including the human-in-the-loop optimisation provided by ALAO.
- **External Organizations / Conservation Agencies** can audit the transparent, continuously improving loop that turns crowd-sourced labels into higher-quality ecological insights.

4.1.2 Component Diagram

The updated architecture now consists of **seven** key subsystems (Figure 4.1). The additional subsystem, ALAO, creates a closed active-learning loop that automatically selects uncertain predictions for re-annotation, stores the verified labels, and triggers retraining of the models.

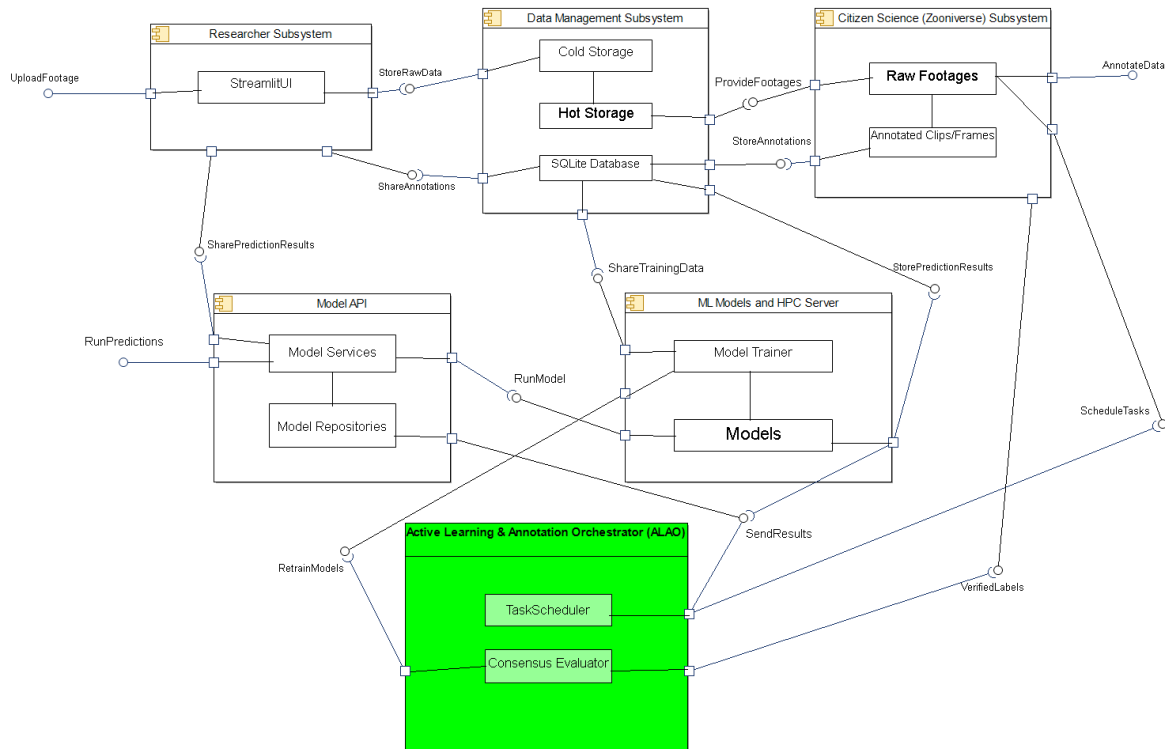


Figure 4.1: Revised component diagram of the KSO system, extended with the *Active-Learning & Annotation Orchestrator (ALAO)*.

Component Descriptions (revised):

- **Researcher Subsystem** – Web UI where researchers upload raw footage, trigger predictions, and review results (`UploadFootage()`, `SharePredictionResults()`).
- **Data Management Subsystem** – Cold or hot object storage plus SQLite meta-data store; supplies clips on demand and saves annotations (`StoreRawData()`, `ProvideFootages()`, `StoreAnnotations()`, `ShareTrainingData()`).
- **Citizen Science (Zooniverse) Subsystem** – Receives clip batches for volunteer annotation and returns consensus labels (`AnnotateData()`, `VerifiedLabels`).
- **Active-Learning & Annotation Orchestrator (ALAO)** – Bridges ML output and human input. *TaskScheduler* picks low-confidence samples and sends them to Zooniverse (`ScheduleTasks`); *ConsensusEvaluator* ingests verified labels, stores them, and signals the ML server to retrain (`RetrainModels`).
- **ML Models and HPC Server** – Trains and serves detection models; publishes prediction scores and listens for retrain triggers (`RunModel()`, `Predictions`, `RetrainModels`).
- **Model API** – Public gateway that executes inference requests and delivers prediction results (`RunPredictions()`, `SharePredictionResults()`).

4.1.3 Internal Interfaces

This section should include an **Internal Interfaces Class Diagram** for your suggestions. Descriptions of the operations given in the internal interfaces class diagram should also be provided. **You should aim for 2 internal interfaces.**

This section defines two core internal interfaces designed to support the smart automation and data management architecture of the KSO system.

SmartAnalysisInterface This interface unifies the operations of species recognition, model evaluation, training, and alert generation. It acts as the intelligent analysis layer in the system.

Attributes

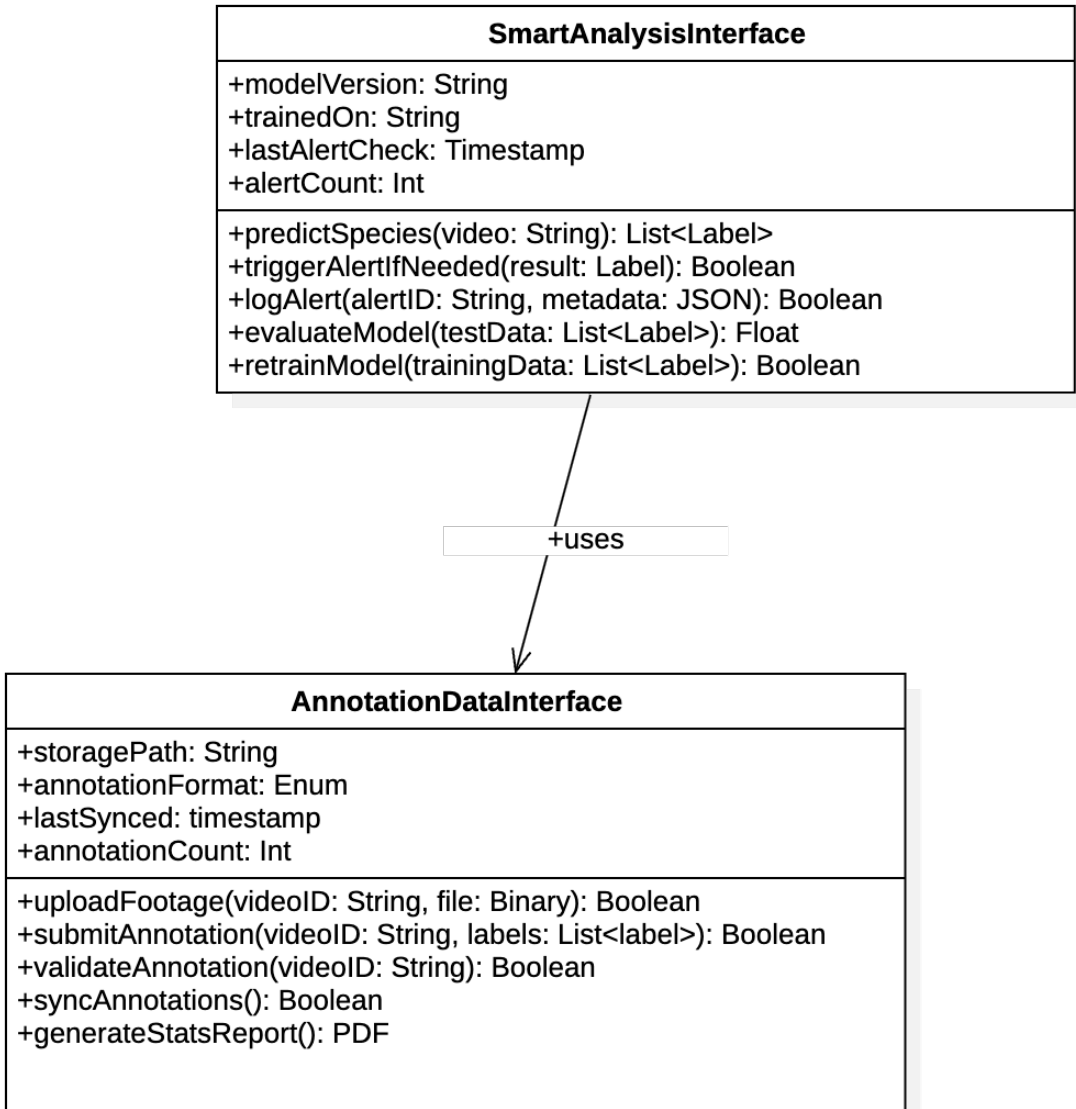


Figure 4.2: Revised Internal Interfaces of the KSO system

- `modelVersion: String` – identifier for the current ML model
- `trainedOn: String` – description of the training data
- `lastAlertCheck: Timestamp` – time of the last alert scan
- `alertCount: Int` – number of alerts generated

Operations

- `predictSpecies(video)` – processes video and returns detected species
- `triggerAlertIfNeeded(result)` – checks if alert conditions are met and triggers if needed
- `logAlert(alertID, metadata)` – stores alert with metadata
- `evaluateModel(testData)` – evaluates model accuracy
- `retrainModel(trainingData)` – updates the model with new training data

AnnotationDataInterface This interface combines annotation-related features and data storage logic. It manages the video upload process, user annotations, and synchronization tasks.

Attributes

- `storagePath: String` – file system location of stored data
- `annotationFormat: Enum` – supported annotation formats (e.g., JSON, XML)
- `lastSynced: Timestamp` – last successful synchronization
- `annotationCount: Int` – number of annotations stored

Operations

- `uploadFootage(videoID, file)` – uploads raw video footage
- `submitAnnotation(videoID, labels)` – stores user-submitted species labels

- `validateAnnotation(videoID)` – ensures that annotations meet formatting and content standards
- `syncAnnotations()` – synchronizes data to backup and ML pipelines
- `generateStatsReport()` – creates a visual or tabular summary of current annotation status

4.1.4 Interaction Patterns

This section includes **1 Sequence Diagram** to show messaging sequences taking place among the system components over the **internal interfaces for your suggestions**. Choose the most complex interaction to model with sequence diagram.

4.1.5 External Interfaces

The original KSO platform exposed three public interfaces (Streamlit UI, Zooniverse API, and the FastAPI Model Gateway), plus an internal Data-Management API. To support the new active-learning feedback loop we introduce **two additional external interfaces** (Figure 4.2):

- **ActiveLearningTask API** – provided by the *Active-Learning & Annotation Orchestrator (ALAO)*; batches uncertain clips and exchanges annotation results with Zooniverse.
- **ModelLifecycle Webhook** – provided by the *ML Models & HPC Server*; streams low-confidence predictions to ALAO and receives retrain notifications.

Interface and Operation Descriptions

ActiveLearningTask API (ALAO ↔ Zooniverse)

- `+ create_task_batch(in clips: list): string` – returns a `batch_id` for the new task bundle.

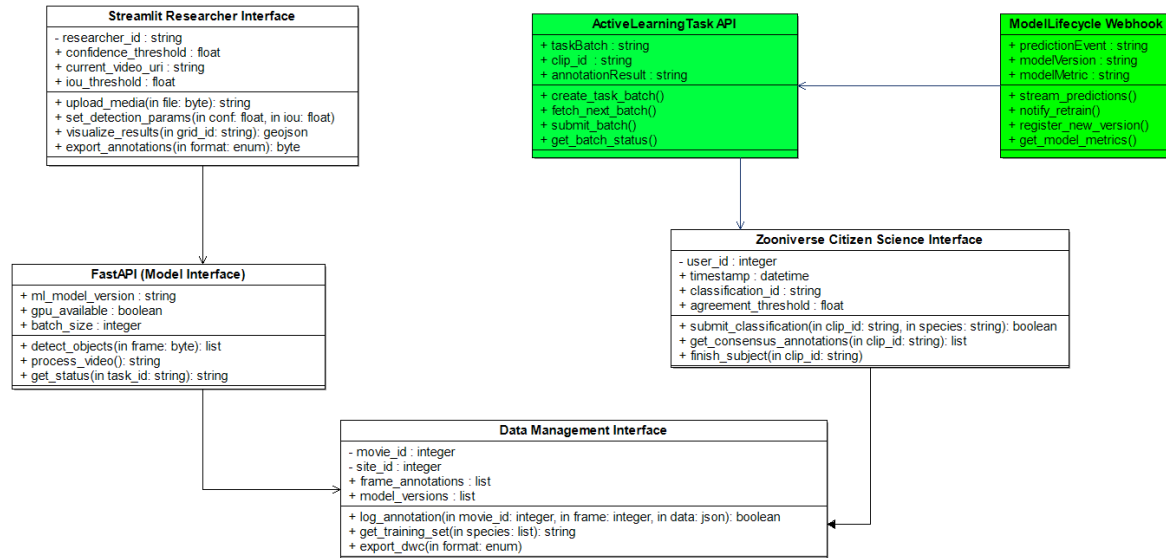


Figure 4.3: Updated External Interfaces class diagram including the two new APIs (highlighted).

- `+ fetch_next_batch(in user_id: int): list` – delivers the next clip list assigned to a volunteer.
- `+ submit_batch(in batch_id: string, in annotations: json): bool` – writes volunteer labels and returns `true` on success.
- `+ get_batch_status(in batch_id: string): string` – returns *queued* / *in_review* / *done*.

ModelLifecycle Webhook (ML Server ↔ ALAO)

- `+ stream_predictions(in clip_id: string, in preds: json): bool` – pushes low-confidence detections to ALAO.
- `+ notify_retrain(in model_id: string): bool` – acknowledges a retrain trigger issued by ALAO.
- `+ register_new_version(in model_id: string, in ver: string): bool` – records freshly trained model artefacts.
- `+ get_model_metrics(in model_id: string): json` – returns accuracy,

F1, mAP, and related metrics for the given model.

Together, these two APIs close the human-in-the-loop cycle:

1. The ML Server streams uncertain predictions via `stream_predictions`.
2. ALAO packages those clips with `create_task_batch` and sends them to Zooniverse.
3. Volunteer labels return through `submit_batch` and are persisted by the Data-Management subsystem.
4. After a quota is reached, ALAO calls `notify_retrain`; the ML Server trains a new model and registers it via `register_new_version`.

4.1.6 Interaction Scenarios

This section presents an activity diagram illustrating the most complex runtime interaction introduced by our architectural suggestions—namely, the active learning feedback loop that spans new external interface. This interaction was not previously covered in the SRS document and represents a critical enhancement in enabling automated, iterative model improvement through human-in-the-loop workflows.

Interaction Steps:

- **1. Generate Predictions:** The ML Model Interface processes newly ingested footage to generate prediction results using existing trained models.
- **2. Filter Low Confidence:** Predictions are filtered to isolate low-confidence outputs that may benefit from human review.
- **3. Stream Predictions:** These uncertain predictions are streamed to the ALAO component via the `stream_predictions()` method on the ModelLifecycle Webhook.

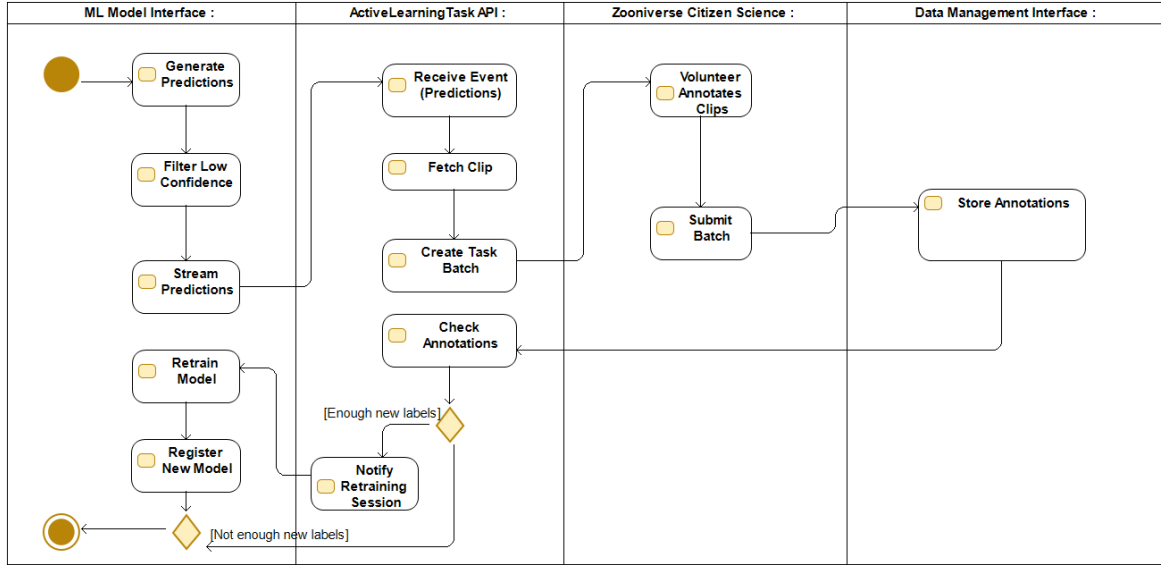


Figure 4.4: Activity diagram of the Active Learning feedback loop spanning ML Model Interface, ALAO, Zooniverse, and Data Management components.

- **4. Receive Event & Fetch Clip:** ALAO receives the event, retrieves the associated video clip from the Data Management Interface, and prepares it for re-annotation.
- **5. Create Task Batch:** A new annotation task batch is created using the `create_task_batch()` method from the ActiveLearningTask API and assigned to the Zooniverse platform.
- **6. Volunteer Annotates Clips:** Citizen scientists annotate the clips via Zooniverse, contributing new labels for model improvement.
- **7. Submit Batch:** Completed annotation batches are returned to ALAO using the `submit_batch()` operation.
- **8. Store Annotations:** ALAO forwards the verified annotations to the Data Management Interface, ensuring permanent storage and traceability.
- **9. Check Annotations:** ALAO monitors the number of newly submitted, high-confidence annotations.

- **10. Retrain Decision:** If enough new labels have accumulated, ALAO triggers the model update workflow via `notify_retrain()`.
- **11. Retrain Model:** The ML interface initiates model retraining with the updated data and stores the new model version using `register_new_version()`.
- **[Loop or Exit]:** If not enough labels exist, the system exits and waits for further contributions before retraining.

Design Implications: This interaction illustrates how active learning introduces a closed loop between prediction, human verification, and retraining—reducing human effort, increasing model accuracy, and ensuring data quality across the system.

4.1.7 Database Class Diagram

Figure 4.4 shows the updated database class diagram for the KSO system, incorporating the new entities added as part of our architectural suggestions. These include **Alert**, **Notification**, **Message**, **EcoHealth**, and **Organizations**, which are highlighted in green.

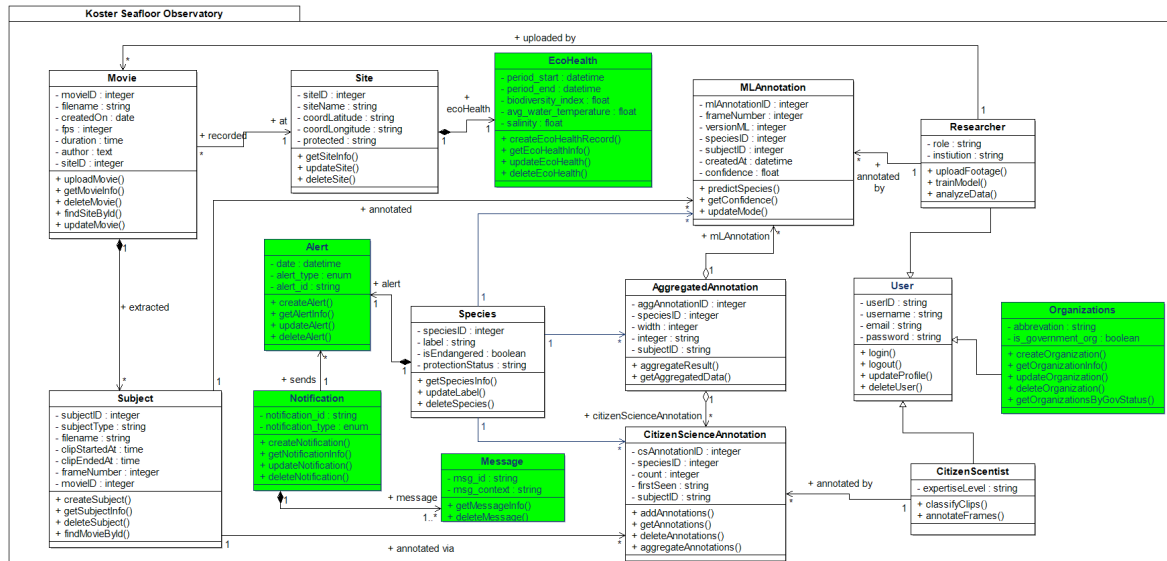


Figure 4.5: Updated database class diagram with new suggested entities highlighted in green.

Description of Newly Added Entities:

- **Alert:** Captures alert events triggered by ecological anomalies or system-defined thresholds. Each alert includes a timestamp, type (e.g., biodiversity drop), and identifier. Alerts are often linked to notification messages and site locations.
- **Notification:** Represents user-facing alert messages or system-generated warnings. Each notification is tied to an alert and has a type (email, in-app, SMS).
- **Message:** Contains predefined or dynamically generated message templates used by the notification system. This ensures reusable, centralized alert messages.
- **EcoHealth:** Stores environment-level metrics tied to specific marine sites over time. Attributes include biodiversity index, water temperature, salinity, and time intervals.
- **Organizations:** Represents institutional affiliations or stakeholders such as research institutes, NGOs, or governmental bodies. Includes flags for governmental status.

4.1.8 Operations on Data

Each of the newly added entities supports standard CRUD operations, as detailed below:

- **Alert**
 - `createAlert()` – Insert a new alert entry for a monitored event.
 - `getAlertInfo()` – Retrieve alert details by ID.
 - `updateAlert()` – Modify the alert type or metadata.
 - `deleteAlert()` – Permanently remove the alert.
- **Notification**

- `createNotification()` – Create a new notification message for a user or group.
 - `getNotificationInfo()` – Fetch notification content and status.
 - `updateNotification()` – Edit delivery type or target users.
 - `deleteNotification()` – Delete expired or acknowledged notifications.
- **Message**
 - `createMessage()` – Store a new message template.
 - `getMessageInfo()` – Retrieve a message based on context or ID.
 - `deleteMessage()` – Remove old or unused messages.
- **EcoHealth**
 - `createEcoHealthRecord()` – Insert a new site-level ecohealth record.
 - `getEcoHealthInfo()` – Fetch metrics for a given site and date range.
 - `updateEcoHealth()` – Modify measurement values like temperature or biodiversity index.
 - `deleteEcoHealth()` – Remove outdated or incorrect ecohealth entries.
- **Organizations**
 - `createOrganization()` – Add a new organization (e.g., NGO, research group).
 - `getOrganizationInfo()` – Retrieve details including name and gov-status.
 - `updateOrganization()` – Edit organizational data.
 - `deleteOrganization()` – Delete an organization entity.
 - `getOrganizationsByGovStatus()` – Filter organizations based on government affiliation.

4.2 Development View

Refer to *R&W Chapter 20*.

4.2.1 Stakeholders' Uses of This View

4.2.2 Package Diagram

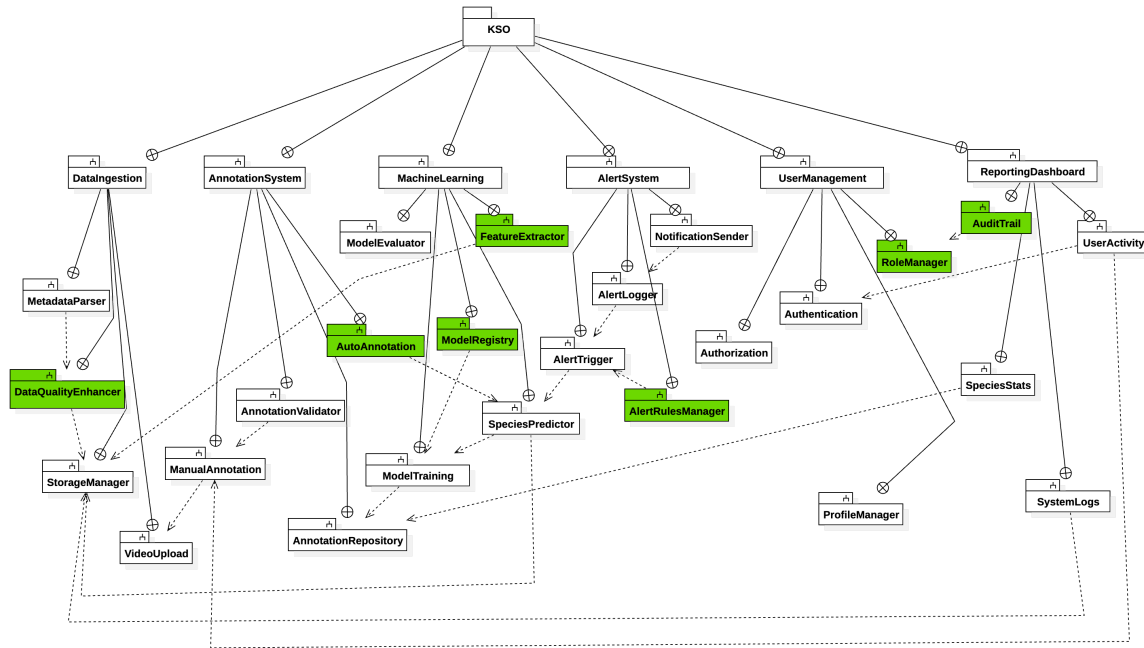


Figure 4.6: Updated package diagram with new suggested entities highlighted in green.

Descriptions of Suggested Components for System Enhancement

In order to increase the scalability, maintainability, and automation of the KSO system, seven new components have been proposed and added to the existing package architecture. These components address key concerns such as data quality, annotation automation, model management, and user governance. Their roles and interactions are described as follows:

- **DataQualityEnhancer (within DataIngestion)**

This module is responsible for enforcing strict data validation rules before video

and metadata are accepted by the system. It includes automated checks for missing metadata, corrupted video formats, resolution consistency, and timestamp integrity. By performing these verifications early in the pipeline, it prevents downstream processing errors and ensures only high-quality data is passed to storage or machine learning modules. This component directly contributes to the reliability and scientific integrity of KSO datasets.

- **AutoAnnotation (within AnnotationSystem)**

The AutoAnnotation component enables semi-automated labeling of marine species in uploaded footage using pre-trained models. It generates initial suggestions that are either accepted or corrected by human annotators. This hybrid approach reduces the manual workload for researchers and citizen scientists while also accelerating dataset generation for model training. It integrates tightly with the AnnotationValidator for consistency checks.

- **ModelRegistry (within MachineLearning)**

A centralized repository for managing the lifecycle of machine learning models. It tracks versions, training parameters, evaluation metrics, and deployment statuses of all models used in the system. The registry supports rollback to previous model versions and enables reproducibility in experiments. It is particularly important for governance in scientific applications, where traceability of decision-making models is crucial.

- **FeatureExtractor (within MachineLearning)**

This component preprocesses raw video and metadata to extract structured features suitable for model training and inference. It applies transformations such as frame selection, object detection preprocessing, statistical summarization, and temporal analysis. The extracted features are stored or passed to the model training and prediction modules. FeatureExtractor plays a key role in improving ML model performance and reducing computation costs.

- **AlertRulesManager (within AlertSystem)**

A rule engine that defines and manages alerting policies used in the AlertSystem. For instance, it can trigger notifications when rare or invasive species are detected, or when large annotation discrepancies occur. Rules can be configured via the admin dashboard, enabling dynamic alert thresholds. It enhances the responsiveness of the system to real-time marine events.

- **RoleManager (within UserManagement)**

RoleManager implements Role-Based Access Control (RBAC) to define user types such as "Researcher", "Citizen Scientist", and "Admin". It determines what operations and datasets each role can access. This supports scalable and secure governance of data access, ensuring compliance with privacy and research policies. It integrates with the Authentication and Authorization modules to enforce policies consistently.

- **AuditTrail (within ReportingDashboard)**

This component logs all critical actions taken in the system, including video uploads, annotations, model deployments, alert modifications, and user access events. It provides a chronological record for accountability and debugging. AuditTrail is especially useful in scientific and regulatory contexts where transparency and reproducibility are required. It is stored securely and can be queried by administrators for monitoring or legal auditing.

Together, these enhancements significantly increase the robustness, automation, and observability of the KSO architecture. They also align the system more closely with best practices in machine learning operations (MLOps), citizen science validation, and cloud-native software design.

This section should include a **Package Diagram** showing the higher-level software modules and their dependencies **for your suggestions**. Package decomposition hierarchy must involve 3 levels, namely, (level 0) the system (kso), (level 1) subsystems of kso, and (level 2) sub-subsystem levels. **Briefly explain each package's contents and show some dependencies between the packages.**

4.3 Deployment View

Refer to *R&W Chapter 21*.

4.3.1 Stakeholders' Uses of This View

4.3.2 Deployment Diagram

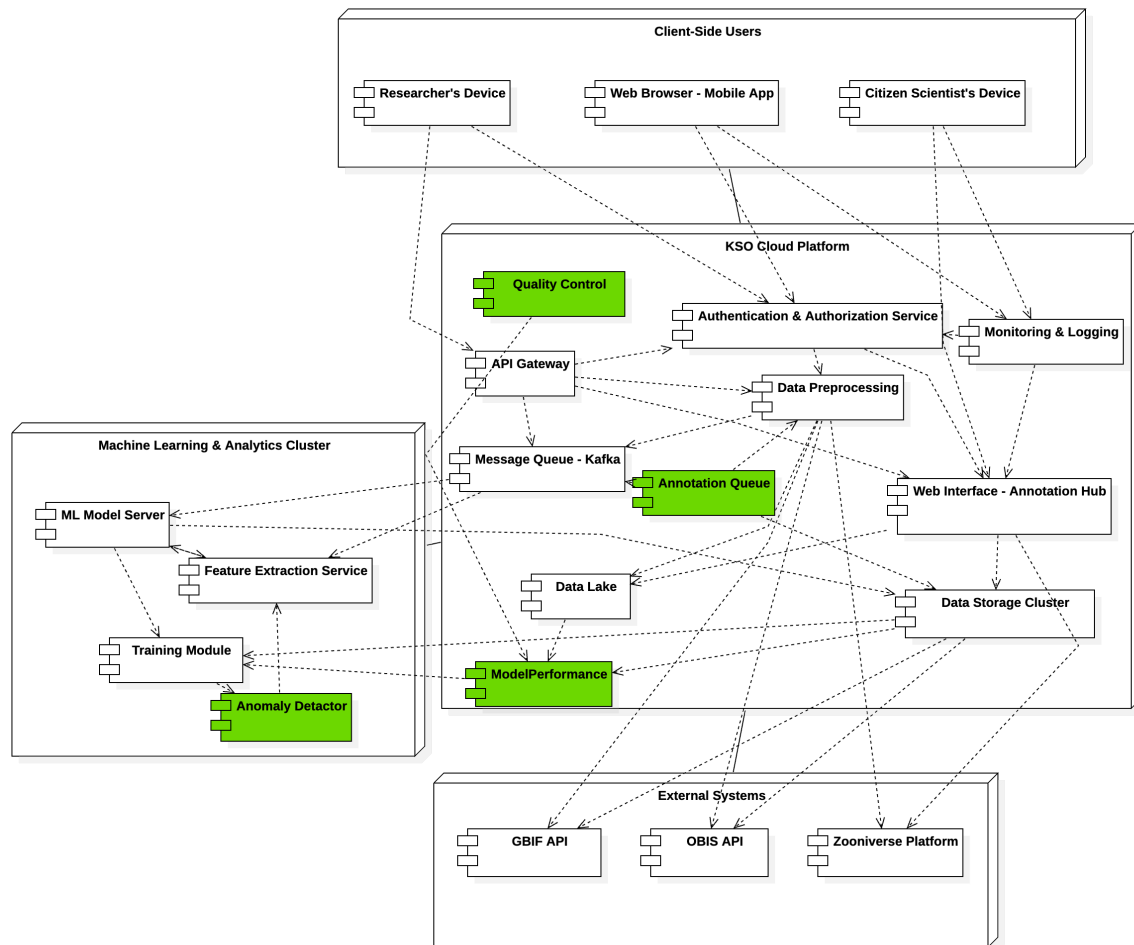


Figure 4.7: Updated deployment diagram with new suggested entities highlighted in green.

New Components

Quality Control Service

- **Purpose:** Automated validation of citizen scientist annotations against ML predictions
- **Function:**
 - Compares human and ML labels using consensus algorithms
 - Flags low-confidence annotations (<0.85) for review
 - Maintains annotation quality standards
- **Tech:** Python + Scikit-learn rules engine

Annotation Queue

- **Purpose:** Manages distribution of annotation tasks
- **Features:**
 - Redis-based priority queue
 - Gamification (user skill-based task assignment)
 - Real-time progress tracking
- **Data:** JSON task objects with timeouts

Model Performance Dashboard

- **Purpose:** Monitors ML model health
- **Metrics:**
 - Precision/recall by species
 - Data drift detection
 - Inference latency tracking
- **Visualization:** Grafana with Prometheus backend

Anomaly Detector

- **Purpose:** Identifies rare marine species
- **Algorithms:**
 - Isolation Forest for unsupervised detection
 - Few-shot learning for new species
- **Thresholds:** 0.9 anomaly score triggers alerts

Key Interactions Quality Control → Model Performance

- **Purpose:** Feedback loop for model improvement
- **Protocol:** gRPC with Protobuf
- **Data:** Validation metrics (FPs/FNs) every 5s
- **Impact:** Triggers retraining when F1 drops $\geq 10\%$

Annotation Queue - Message Queue

- **Purpose:** Task distribution pipeline
- **Protocol:** Kafka topics (10 partitions)
- **Data:** Prioritized image batches
- **Throughput:** 1MB/s sustained

Anomaly Detector - Feature Extraction

- **Purpose:** Rare species identification
- **Data Flow:** 4096-dim embedding vectors
- **Latency:** ≤ 100 ms per detection
- **Alerting:** Webhooks to monitoring

This section should include a **Deployment Diagram and the explanations** for the deployment diagram to show a possible allocation of components over resources **for your suggestions**.

4.4 Design Rationale

State **one design rationale** specifically referring to each view you have presented for **your suggestions**.